

STUDENT VIVA & CODE COMPANION

Case Study No. 118: Movie Review Platform | Semester 3 Backend Development

Student Confidential Guide

CineReview: Everything Explained Simply

Author: Aaroh Wankhade | Student ID: 150096726175 | University: ITM Skills University

The "Explain Like I'm 5" Big Picture Analogy

Imagine a digital movie theater noticeboard called **CineReview**:

- Browsing (Public):** Anyone walking on the street can look at the movie posters and read what others wrote.
- Writing a Review (Requires Ticket/JWT):** If you want to post your own score (1 to 5 stars) and review, you must first register or log in. When you log in, the server stamps a VIP wristband called a **JWT Token** onto your browser.
- The Math Teacher (MongoDB Aggregation \$avg):** Every time someone gives a rating, MongoDB automatically takes all ratings for that movie, adds them up, divides by the total count, and shows the average score (e.g., 4.7 / 5.0) in real-time.
- The Security Guard (Ownership Middleware):** You can only edit or throw away (delete) *your own* review. If another student tries to edit your review, the server shouts **HTTP 403 Forbidden!** and stops them instantly.

1. The Exact 5 Case Study Objectives & How We Solved Them

Case Study Objective	How It Works (In Plain English)	File Responsible
1. Mongoose Schemas with References	Movies and Reviews are separate documents. The Review document stores the ObjectId of the Movie and User it belongs to.	<code>models/Movie.js</code> <code>models/Review.js</code>
2. CRUD Operations	Create, Read, Update, Delete for movies and audience reviews via standard HTTP methods (POST, GET, PATCH, DELETE).	<code>controllers/movieController.js</code> <code>controllers/reviewController.js</code>
3. Ownership Authorization	Checks if the logged-in user's ID matches the author's ID on the review. If they don't match, blocks with 403 Forbidden .	<code>middleware/ownershipMiddleware.js</code>
4. Dynamic Average Aggregation	Runs a 2-stage MongoDB pipeline: \$match filters reviews for the movie, and \$group with \$avg computes the mathematical mean.	<code>controllers/movieController.js</code> (Endpoint: <code>/average-rating</code>)
5. Strict Validation	Ensures ratings are integers between 1 and 5, review text is non-empty, and required fields are never omitted.	<code>middleware/validationMiddleware.js</code> <code>models/Review.js</code>

What Makes This Backend Pure & Syllabus-Compliant?

Zero external bloat. We used only Node.js, Express.js, MongoDB/Mongoose, JWT, and Bcryptjs. Everything asked in the Case Study is present, and nothing extra is cluttering your project.

ARCHITECTURE MAP: WHAT IS DONE WHERE?

Complete Directory Breakdown for Quick Viva Reference

Clean & Minimal Architecture

2. Backend Project Structure & File Duties

When the examiner asks: "Where does this happen?", use this cheat sheet:

Directory & File	What It Does (Plain Terms)	Examiner Key Takeaway
backend/server.js	The front door of our application. Sets up Express, CORS, static file serving, connects to MongoDB, and listens on port 5001.	Initial entry point; loads environment and mounts all API routes.
backend/config/db.js	Database connection manager. Connects to MongoDB Atlas URI or starts automated in-memory MongoDB fallback.	Zero external server dependency; runs standalone anywhere.
backend/models/User.js	User blueprint. Has name, email, password. Automatically hashes passwords with bcrypt before saving.	Bcrypt pre-save hook; matches passwords securely.
backend/models/Movie.js	Movie blueprint. Stores title, genre, releaseYear, posterUrl, and createdBy.	Holds movie metadata with validation.
backend/models/Review.js	Review blueprint. Stores movie (ObjectId), user (ObjectId), rating (1-5), and reviewText .	Mongoose References linking collections together.
middleware/authMiddleware.js	Inspects the incoming HTTP Authorization: Bearer <token> header. Unpacks JWT and attaches req.user .	Protects private routes from unauthenticated users (401).
middleware/ownershipMiddleware.js	Compares review.user.toString() === req.user._id.toString() . Blocks non-authors with HTTP 403 Forbidden .	Core Case Study Objective 3! Ownership guard.
middleware/validationMiddleware.js	Validates that rating is a number from 1 to 5, and review text is not empty or too short before touching the database.	Core Case Study Objective 5! Prevents bad data.
controllers/movieController.js	Handles movie queries and the MongoDB Aggregation Pipeline to compute dynamic average ratings.	Core Case Study Objective 4! Uses \$match and \$group .
controllers/reviewController.js	Creates reviews, retrieves reviews for a movie, and handles owner-only updates and deletions.	Core Case Study Objective 2! CRUD logic.

3. Request Lifecycle Diagram: What Happens When User Hits "Publish"?

[Frontend Button Click]
↓ Sends POST /api/movies/:id/reviews with Bearer Token & JSON { rating: 5, reviewText:

```
"Great!" }  
[authMiddleware.js]  
  ↓ Verifies JWT signature. Valid? Attaches req.user. Invalid? Returns 401 Unauthorized.  
[validationMiddleware.js]  
  ↓ Checks: Is rating 1-5? Is text present? Valid? Calls next(). Invalid? Returns 400 Bad  
Request.  
[reviewController.js]  
  ↓ Executes Review.create({ movie: movieId, user: req.user._id, rating: 5, reviewText: "..."  
})  
[Database Updated & Aggregation]  
  ↓ Review saved! MongoDB recalculates average rating via $avg pipeline. Returns 201 Created  
to UI.
```

LINE-BY-LINE CODE EXPLAINER

The Exact Lines Every Examiner Asks About, Explained Simply

Key Viva Code Snippets

Snippet 1: The MongoDB Aggregation Pipeline (\$avg, \$match, \$group)

Location: `backend/controllers/movieController.js`

```
const stats = await Review.aggregate([
  // Stage 1: Filter only reviews that belong to THIS specific movie
  {
    $match: { movie: new mongoose.Types.ObjectId(movieId) }
  },
  // Stage 2: Group all matching reviews together to calculate the math
  {
    $group: {
      _id: '$movie', // Group key is the movie ID
      averageRating: { $avg: '$rating' }, // Computes mathematical average of ratings
      totalReviews: { $sum: 1 } // Counts 1 for each review document
    }
  }
]);
```

Line-by-Line Explanation for Examiner:

- `Review.aggregate([...])` : Runs MongoDB's native multi-stage data processing pipeline instead of manually looping in Node.js.
- `$match` : Like an SQL `WHERE` clause. It quickly filters only reviews where `movie == movieId`.
- `$group` : Like an SQL `GROUP BY`. It collapses all matched reviews into a single summary record.
- `$avg: '$rating'` : MongoDB calculates the exact mean rating directly inside the database engine for maximum speed.
- `$sum: 1` : Increments a counter by 1 for every document, giving us the total review count.

Snippet 2: The Ownership Authorization Middleware (403 Forbidden)

Location: `backend/middleware/ownershipMiddleware.js`

```
const authorizeReviewOwner = async (req, res, next) => {
  const review = await Review.findById(req.params.id);
  if (!review) {
    return res.status(404).json({ message: 'Review not found' });
  }

  // Compare author ID with logged-in user ID
  const isOwner = review.user.toString() === req.user._id.toString();

  if (!isOwner) {
    return res.status(403).json({
      success: false,
      message: 'Forbidden: You are only authorized to edit or delete your own reviews'
    });
  }

  req.review = review; // Attach review to request object for controller use
  next(); // Allow request to continue!
};
```

Line-by-Line Explanation for Examiner:

- `review.user.toString() === req.user._id.toString()` : *Why `.toString()` ?* In MongoDB, IDs are binary `ObjectId` objects. Comparing two objects with `===` checks memory references, not values! Converting to string ensures we compare the actual 24-character hexadecimal IDs.
- **HTTP 403 Forbidden** : The official HTTP standard status code indicating that the server knows who the user is, but refuses to authorize the action.

Snippet 3: Bcrypt Password Hashing Pre-Save Hook

Location: `backend/models/User.js`

```
userSchema.pre('save', async function (next) {  
  if (!this.isModified('password')) return next();  
  const salt = await bcrypt.genSalt(10);  
  this.password = await bcrypt.hash(this.password, salt);  
  next();  
});
```

Explanation: Automatically scrambles plain passwords into irreversible hashes with 10 salt rounds before saving to database. Even if the database is leaked, raw passwords are never compromised.

TOP VIVA QUESTIONS & SPOKEN ANSWERS

Memorize or Review These Bullet Points for High Scoring

100% Viva Success

4. Quick Viva Q&A Cheat Sheet (Speak These Aloud)

Q1: How do you establish relationships between Movie and Review in Mongoose?

Answer: "We use Mongoose document referencing. In the Review schema, we define `movie: { type: mongoose.Schema.Types.ObjectId, ref: 'Movie' }` and `user: { type: mongoose.Schema.Types.ObjectId, ref: 'User' }`. When querying, we can use `.populate('user', 'name email')` to join document data dynamically without duplicating records."

Q2: Why compute the average rating using MongoDB Aggregation instead of JavaScript loops?

Answer: "Calculating averages inside JavaScript requires fetching all reviews into Node.js memory, causing memory spikes and network latency when there are thousands of reviews. The MongoDB Aggregation pipeline (`$match` and `$group` with `$avg`) executes directly in C++ on the database engine and returns only the single aggregated number."

Q3: What is the difference between 401 Unauthorized and 403 Forbidden?

Answer: "**401 Unauthorized** means authentication is missing or invalid—the server doesn't know who you are (e.g. missing JWT). **403 Forbidden** means authentication succeeded, but you do not have permission to modify this specific resource (e.g. User A trying to delete User B's review)."

Q4: How does JWT authentication work in your application?

Answer: "When a user logs in, the server generates a signed JSON Web Token using `jwt.sign({ id: user._id }, JWT_SECRET)`. The frontend stores this token and sends it in the HTTP `Authorization: Bearer <token>` header. Our `protect` middleware parses the token using `jwt.verify()` and attaches the user object to `req.user`."

Q5: What happens if a user submits a rating of 6 or -1?

Answer: "It is rejected before touching the database by our `validateReviewInput` middleware with **HTTP 400 Bad Request**. Additionally, the Mongoose schema has `min: 1` and `max: 5` validators to ensure database-level schema integrity."

Q6: Why did you convert IDs to strings using `.toString()` in ownership check?

Answer: "MongoDB `_id` fields are BSON ObjectId objects. Comparing two objects with `===` checks if they point to the same memory location, which evaluates to false. Calling `.toString()` ensures we compare their 24-character hexadecimal values."

Q7: How did you implement cascade deletion for movies?

Answer: "In `movieController.js` under `deleteMovie`, before deleting the movie document, we run `await Review.deleteMany({ movie: movie._id })` to eliminate orphan reviews and keep the database consistent."

STEP-BY-STEP GITHUB & CLOUD DEPLOYMENT GUIDE

How to Push Code, Deploy, and Get Live Links for Case Study Deliverable

Deliverables Step-by-Step

5. Case Study Deliverable: Step-by-Step Deployment Guide

The case study states: "Deploy backend on Render/Railway with MongoDB Atlas; deploy frontend on Vercel/Netlify." Here is the exact recipe:

Part A: Push Your Project to GitHub (5 Minutes)

1. Open your terminal in the project folder:

```
cd "/Users/aarohwankhade/Desktop/Backend Project"
```

2. Create a new repository on your GitHub account (github.com/new) named: `movie-review-platform` (set to Public).
3. Link and push your local commits to GitHub:

```
git remote add origin https://github.com/YOUR_GITHUB_USERNAME/movie-review-platform.git
git branch -M main
git push -u origin main
```

4. Your GitHub Repository Link: `https://github.com/YOUR_GITHUB_USERNAME/movie-review-platform`

Part B: Setup Free MongoDB Atlas Database (3 Minutes)

1. Sign in to [MongoDB Atlas](https://www.mongodb.com/atlas) and create a Free M0 Shared Cluster.
2. Under **Database Access**, create a user (e.g. username: `cinereview_admin`, password: `SecurePass123!`).
3. Under **Network Access**, click **Add IP Address** and select **Allow Access From Anywhere (0.0.0.0/0)**.
4. Click **Connect** → **Drivers** → Copy the Connection String:

```
mongodb+srv://cinereview_admin:SecurePass123!@cluster0.abcd.mongodb.net/cinereview_db?
retryWrites=true&w=majority
```

Part C: Deploy Backend on Render (Free & Automatic)

1. Sign in to [Render.com](https://render.com) and click **New +** → **Web Service**.
2. Connect your GitHub repository: `movie-review-platform`.
3. Configure settings:
 - **Root Directory:** `backend`
 - **Build Command:** `npm install`
 - **Start Command:** `node server.js`
4. Under **Environment Variables**, add:
 - `PORT = 5001`
 - `NODE_ENV = production`
 - `MONGO_URI = Your Atlas Connection String from Part B`
 - `JWT_SECRET = supersecret_itm_semester3_jwt_key_2026`
5. Click **Deploy Web Service**. In 2 minutes, Render gives you your live URL:

```
https://movie-review-platform-api.onrender.com
```

Part D: Deploy Frontend on Vercel / Netlify

1. In `frontend/js/config.js`, update `API_BASE_URL`:

```
const API_BASE_URL = 'https://movie-review-platform-api.onrender.com/api';
```

2. Commit and push the change to GitHub:

```
git commit -am "chore: point frontend to production Render backend" && git push
```

3. Sign in to [Vercel](#), click **Add New Project**, select `movie-review-platform`, set Root Directory to `frontend`, and click **Deploy**!

4. **Your Live Deployable Frontend URL:** `https://movie-review-platform.vercel.app`

Summary of Deliverables to Submit to University:

1. **GitHub Repository URL:** `https://github.com/YOUR_USERNAME/movie-review-platform`
2. **Live Frontend URL:** `https://movie-review-platform.vercel.app`
3. **Live REST API URL:** `https://movie-review-platform-api.onrender.com/api`
4. **Project Report PDF:** `Movie_Review_Platform_Report.pdf` (14 pages)
5. **Student Viva Companion PDF:** `STUDENT_VIVA_AND_CODE_EXPLAINER.pdf` (5 pages)